

Week 5 Assignment

Apache Spark Fundamentals and Data Cleaning Using DataFrames

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

MapReduce

1. Stores intermediate results on disk after every stage, making processing slower.
2. It is not efficient for iterative machine learning tasks and requires multiple jobs for complex workflows.

Spark

1. It performs computations in memory, resulting in faster execution and better performance.

Spark is preferred because it provides faster processing, supports real-time analytics, machine learning, SQL operations, and distributed computing.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

1. Spark stores intermediate data in RAM instead of repeatedly reading and writing data from disk.
2. Machine learning algorithms often process the same dataset multiple times.
3. Keeping data in memory significantly reduces execution time.

In-memory computing improves performance and reduces disk I/O overhead.

Q3. Remove duplicate rows based on user_id and transaction_date.

Q3 Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

```
[13]
✓ 3s  print("Rows Before:", df.count())

      df_no_duplicates = df.dropDuplicates(
        ["user_id", "transaction_date"]
      )

      print("Rows After:", df_no_duplicates.count())
```

▼
... Rows Before: 110
Rows After: 100

Duplicate transaction records were removed using user_id and transaction_date as unique identifiers.

Q4. Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Code

Q4 Write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
[16]
✓ 1s
from pyspark.sql.functions import avg

df.filter(
    col("region") == "West"
).groupBy(
    "product_category"
).agg(
    avg("sale_amount").alias("avg_sale")
).show()
```

```
+-----+-----+
|product_category|      avg_sale|
+-----+-----+
|      Electronics|2055.0025000000005|
|      Clothing|2491.7855555555557|
|        Books|2908.5600000000004|
|      Furniture|      1478.89|
+-----+-----+
```

Average sales were calculated for each product category in the West region.

Q5. Difference between .na.drop() and .na.fill(). Provide a code example of filling null values in a status column with the string 'Unknown'.

- **.na.drop()** removes rows containing null values.
- **.na.fill()** replaces null values with a specified value.

Code

Q5 Provide a code example of filling null values in a status column with the string 'Unknown'.

```
[15]
✓ 0s
df_filled = df.na.fill({
    "status": "Unknown"
})

df_filled.show(5)
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|user_id|transaction_date|region|product_category|sale_amount|city|age|subscription|email|username|price|store_id|raw_timestamp|status|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|3765|2025-11-12|North|Clothing|660.18|Pune|26|Basic|Basic@pjacobs@example.org|hugheswilliam|975.1|S104|1972-03-03 19:10:...|Pending|
|7644|2026-03-10|West|Furniture|1572.95|Pune|19|Basic|NULL|karencruz|141.56|S103|1995-08-26 21:08:...|Active|
|2798|2026-01-01|North|Electronics|4365.53|Bangalore|22|Basic|NULL|terri58|794.53|S103|2003-09-10 13:51:...|Inactive|
|2489|2025-10-23|North|Books|4344.35|Mumbai|21|Basic|jmann@example.net|vharvey|1889.22|S104|2005-02-22 01:52:...|Active|
|4222|2026-04-23|West|Books|2506.72|Pune|25|Premium|chall@example.net|NULL|1545.51|S104|2015-07-13 22:09:...|Active|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

only showing top 5 rows

All null values in the status column become "Unknown".

Null values were handled without removing any records.

Q6. Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Code

Q6 Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
[17] ✓ 0s from pyspark.sql.functions import count

city_counts = (
    df.groupBy("city")
      .agg(count("*").alias("total_records"))
)

city_counts.show()
```

```
+-----+-----+
|   city|total_records|
+-----+-----+
|Bangalore|      19|
|Chennai|      13|
|Mumbai|      24|
|Pune|      19|
|Delhi|      35|
+-----+-----+
```

Since the generated dataset contains only 110 records distributed across multiple cities, no city exceeded 100 records. Therefore, the threshold condition was demonstrated conceptually.

Record distribution across cities was analyzed using aggregation.

Q7. How does the immutability of Spark DataFrames affect data cleaning?

Spark DataFrames are immutable, meaning they cannot be changed after creation. Operations like dropping columns, renaming columns, or filling null values create a new DataFrame instead of modifying the original one.

Example:

```
df_new = df.drop("status")
```

Here, `df_new` is a new DataFrame, while the original `df` remains unchanged.

Immutability improves data consistency, fault tolerance, and reliability during data cleaning and processing.

Q8. Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

```
Q8
```

```
[18]
✓ Os df.filter(
      (col("age") >= 18) &
      (col("age") <= 30) &
      (col("subscription") == "Premium")
    ).show()
```

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	email
4222	2026-04-23	West	Books	2506.72	Pune	25	Premium	chall@example.net
7494	2025-06-11	East	Books	2312.17	Delhi	19	Premium	benjamincraig@exa...
3940	2026-01-06	North	Books	788.36	Mumbai	22	Premium	sarah08@example.net
2983	2026-01-23	North	Clothing	4951.87	Delhi	24	Premium	tara46@example.com
6112	2025-08-07	South	Books	3288.52	Delhi	18	Premium	joshuagraham@exam...
8123	2025-07-21	East	Books	1880.76	Mumbai	18	Premium	nancypetty@exampl...
2202	2026-01-15	South	Electronics	985.96	Mumbai	27	Premium	frederickjames@ex...
3822	2026-01-25	South	Furniture	1277.81	Pune	23	Premium	tinaclark@example...
3228	2025-11-16	East	Furniture	1232.9	Delhi	27	Premium	zcardenas@example...
6822	2025-10-06	North	Clothing	4857.94	Delhi	19	Premium	markgrimes@exampl...
3730	2026-02-14	South	Electronics	589.94	Bangalore	22	Premium	chernandez@exampl...
9283	2025-12-24	North	Clothing	226.15	Delhi	20	Premium	johnfitzpatrick@e...
9046	2025-11-17	West	Books	2898.43	Pune	22	Premium	hughesdarren@exam...
1078	2025-07-24	North	Clothing	1144.66	Bangalore	20	Premium	chenjerry@example...
8775	2026-03-25	South	Clothing	288.83	Chennai	23	Premium	susanparker@examp...
6112	2025-08-07	South	Books	3288.52	Delhi	18	Premium	joshuagraham@exam...

The filter operation extracted young premium subscribers.

Q9. When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Null values can produce inaccurate results during calculations such as average, sum, or count. Cleaning missing values before aggregation improves data quality and reliability.

Handling nulls ensures accurate business insights and analysis.

Q10. Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
Q 10
```

```
[23]
✓ Os from pyspark.sql.types import TimestampType
     from pyspark.sql.functions import col
     df = (
         df.withColumn(
             "event_time",
             col("raw_timestamp")
               .cast(TimestampType())
               .drop("raw_timestamp")
         )
     )
```

```
[24] df.printSchema()
✓ Os df.select("event_time").show(5, truncate=False)

... root
|-- user_id: integer (nullable = true)
|-- transaction_date: date (nullable = true)
|-- region: string (nullable = true)
|-- product_category: string (nullable = true)
|-- sale_amount: double (nullable = true)
|-- city: string (nullable = true)
|-- age: integer (nullable = true)
|-- subscription: string (nullable = true)
|-- email: string (nullable = true)
|-- username: string (nullable = true)
|-- price: double (nullable = true)
|-- store_id: string (nullable = true)
|-- status: string (nullable = true)
|-- event_time: timestamp (nullable = true)

+-----+
|event_time|
+-----+
|1972-03-03 19:10:07.581275|
|1995-08-26 21:08:03.816477|
|2003-09-10 13:51:35.389513|
|2005-02-22 01:52:28.875878|
|2015-07-13 22:09:56.277334|
+-----+
only showing top 5 rows
```

The timestamp column was converted into a proper Spark timestamp format.

Q11. Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Shuffle is the process in Spark where data is redistributed across different partitions during operations such as `groupBy()`, `join()`, and `distinct()`. During a grouping operation, records with the same key are moved to the same partition so that Spark can perform the aggregation.

It is considered a **wide transformation** because data from multiple partitions must be exchanged across the cluster. Unlike narrow transformations, where each partition can be processed independently, wide transformations require data movement between executors, which involves network communication and can increase processing time.

Example

```
df.groupBy("city").count().show()
```

In this operation, all records belonging to the same city are shuffled together before counting.

Shuffle is one of the most expensive operations in Spark because it involves transferring data across partitions. Therefore, minimizing unnecessary shuffle operations can significantly improve performance.

Q12. Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

```
[19]
✓ Os  clean_df = df.filter(
    col("email").isNull()
).filter(
    col("username") != ""
)

clean_df.show()
```

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	email
3765	2025-11-12	North	Clothing	660.18	Pune	26	Basic	pjacobs@example.org
2489	2025-10-23	North	Books	4344.35	Mumbai	21	Basic	jmann@example.net
9114	2025-12-25	South	Books	4860.95	Pune	28	Basic	mark81@example.com
6670	2025-09-19	West	Furniture	212.48	Pune	54	Premium	curtis21@example.org
7494	2025-06-11	East	Books	2312.17	Delhi	19	Premium	benjaminrcraig@exa...
8856	2026-04-18	North	Furniture	1703.4	Delhi	29	Basic	brownbrett@exempl...
2892	2026-04-19	North	Books	3665.74	Delhi	25	Basic	sgarcia@example.net
4088	2026-02-13	East	Furniture	138.01	Mumbai	32	Premium	michael07@example...
1036	2025-07-06	South	Books	2728.63	Delhi	25	Basic	kbass@example.net
3787	2025-08-21	North	Books	3188.87	Delhi	54	Premium	icarpenr@exempl...
7072	2026-05-10	West	Furniture	2651.24	Bangalore	32	Premium	alexander79@examp...
3940	2026-01-06	North	Books	788.36	Mumbai	22	Premium	sarah08@example.net
2578	2025-07-18	East	Books	1326.14	Chennai	34	Basic	stevenlopez@examp...
1140	2025-11-24	North	Furniture	3224.59	Pune	41	Premium	garrett73@example...
2792	2025-10-20	North	Clothing	3270.68	Delhi	38	Premium	ocox@example.net
3297	2026-02-11	South	Books	4382.78	Chennai	50	Basic	mary75@example.org
7084	2025-08-16	East	Clothing	2123.01	Mumbai	30	Premium	whurns@example.net

Rows with missing emails or empty usernames are removed.

Invalid records were eliminated to improve dataset quality.

Q13. How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Code

```
Q13

[20]
✓ Os  from pyspark.sql.functions import min, max, avg

df.agg(
    min("price").alias("min_price"),
    max("price").alias("max_price"),
    avg("price").alias("avg_price")
).show()
```

min_price	max_price	avg_price
85.01	1983.82	1075.7460000000003

Multiple statistics were calculated using a single aggregation operation.

Q14. In the context of cleaning a dataset, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

If date formats are inconsistent, inferSchema=true may incorrectly detect the column as a String instead of a Date/Timestamp. This can lead to parsing errors, incorrect sorting, and problems while performing date-based operations.

Example

2025-06-01

01/06/2025

June 1, 2026

For datasets with inconsistent date formats, it is safer to define the schema manually rather than relying on automatic schema inference.

Q15. Write a final processing pipeline that:

- 1. Filters out duplicates.**
- 2. Fills null prices with 0.**
- 3. Groups by store_id to calculate total revenue.**

Q15

```
[21] ✓ 1s from pyspark.sql.functions import sum

final_result = (
    df
    .dropDuplicates()
    .na.fill({"price": 0})
    .groupBy("store_id")
    .agg(
        sum("price").alias("total_revenue")
    )
)

final_result.show()
```

...

store_id	total_revenue
S102	25674.29
S104	30332.010000000002
S101	22088.489999999998
S103	21271.58

The complete pipeline demonstrates duplicate removal, null handling, grouping, and aggregation in a single workflow.

Conclusion

Apache Spark DataFrames were used to perform data cleaning, filtering, schema transformation, grouping, and aggregation. The assignment demonstrated handling duplicates, null values, timestamp conversion, and revenue analysis using distributed data processing concepts. Spark's in-memory architecture makes it significantly faster and more scalable than traditional MapReduce for modern data analytics workloads.